Automatically Identifying FAIR Data and Code Sharing Practices with Metacheck

Daniël Lakens[1], Cristian Mesquida[1], and Lisa DeBruine[2]

[1]Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

[2]Institute of Neuroscience and Psychology, University of Glasgow

Author Note

Daniël Lakens https://orcid.org/0000-0002-8393-5316

Cristian Mesquida https://orcid.org/0000-0002-1542-8355

Lisa DeBruine https://orcid.org/0000-0002-7523-5539

A reproducible version of this manuscript is available at https://github.com/scienceverse/bes_validation. The authors are the developers of the open source tool Metacheck. They have a strong interest in demonstrating that the tool performs well to increase adoption and secure future funding. Thanks to Levi Baruch for his work on the data_check module.

Author roles were classified using the Contributor Role Taxonomy (CRediT; https://credit.niso.org/) as follows: *Daniël Lakens*: conceptualization, writing, funding acquisition. *Cristian Mesquida*: conceptualization, writing. *Lisa DeBruine*: conceptualization, writing

Correspondence concerning this article should be addressed to Daniël Lakens, Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Den Dolech 2, 5600 MB, Eindhoven, Eindhoven, Email: D.Lakens@tue.nl

Abstract

Journals increasingly require authors to share data and code, but checking whether submitted articles comply is effortful. Cooper and the BES Data and Code Hackathon Group (2026) manually coded data- and code-archiving practices in 1861 papers published in the seven British Ecological Society journals between 2017 and 2024, in a two-day hackathon with 145 participants. We examine to what extent this information can be extracted automatically from an article and its linked repositories. We ran Metacheck, an open source tool that checks articles and repositories for best practices, on the same articles and compared its results with the manual coding. Agreement was 85.5% on whether data were available in a repository, 87.9% on whether code was archived, 95.7% on whether the data had a licence, and 72.9% on whether a README was present; where both found data or code, agreement on whether it could be downloaded was above 99%. Claude Opus 5.5 reviewed every disagreement. About two in five of the papers where coders found data or code that Metacheck missed had not shared it in a way a machine can retrieve; most others reflect technical limitations of Metacheck, such as text extraction from PDF files and the repositories and file types it supports. For READMEs, most disagreements reflect a difference in what counts as a README. The manual coding was not always right: for licences, the automatic coding was correct in 46 of 57 disagreements. Metacheck also ran the code of 435 papers: 21.8% of 2874 scripts ran without an error, revealing common problems authors can resolve. Automated tools can find data and code that human coders miss, and could substantially reduce the time needed for such checks. They cannot replace human judgement, but can support workflows that check whether data and code are shared following FAIR principles.

Keywords: Open Science, Data Archiving, Code Archiving, Automation, Metacheck

Automatically Identifying FAIR Data and Code Sharing Practices with Metacheck

The last two decades have seen a notable culture change around open science practices. Universities, journals, and funders increasingly expect researchers to engage in open science practices. One open science practice that has become especially widely required is data and code sharing. The goal of data and code sharing is to facilitate re-use, and to increase transparency. MAny funders have started to require data management plans that specify where and how data and code will be made available for re-use, and submission guidelines for journals often require that data and code underlying the claims in a manuscript are shared if possible. The British Ecological Society, for example, requires, "as a condition for publication, that all data supporting the results in papers published in its journals are archived in an appropriate public archive offering open access and guaranteed preservation", and for theoretical papers that "the underlying model code must be archived" (https://besjournals.onlinelibrary.wiley.com/hub/data_archiving_policy).

Although the requirements to share data and code are clear, checking whether authors follow through on them is difficult. At the BES journals, for example, journal staff check each accepted paper to ensure that data and code are archived, but "do not have the skills or capacity to guarantee" requirements such as completeness (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1955). Manually checking data and code files is a time-consuming task that requires expertise about best practices in data and code sharing. Cooper and the BES Data and Code Hackathon Group (2026b) assessed data- and code-archiving practices across the seven journals published by the British Ecological Society (BES). They found that most manuscripts in which data was used also shared the data, but code sharing was less common, and even when data and code were shared, best practices were often not followed. About a third of data and/or code repositories lacked a README file, and code was not always well-annotated. Studies in other fields show that data and code are often not shared, even when journals require it (Hamilton et al., 2022; Li et al., 2026), and that when code is shared, rerunning it often fails or does not reproduce the reported results (Crüwell et al., 2023; Miske et al., 2026; Obels et al.,

2020; Trisovic et al., 2022). These findings strongly suggest that it would be beneficial if data and code is reviewed before a manuscript is published.

Some journals manually check data and code before an article is published. This practice is well established in economics journals, where a dedicated Data Editor oversees the reproduction of results based on data and code (Vilhuber & Cavanagh, 2025). Manual checks are laborious and delay the publication process. It is therefore not surprising that researchers are exploring automated ways to check whether data and code are shared, and whether they are usable. For example, the Scalable Infrastructure for Validation of Computational Social Science Research (SIVACOR), a project led by the Data Editor of the American Economic Association, examines whether it is possible to automatically verify the computational reproducibility of data and code. The extent to which automated processes can reduce the workload associated with checks of data and code sharing practices is an open question. Automation has the potential to reduce the workload of manual checks, but it is not clear whether and where automated tools can achieve the same level of accuracy as human coders.

In this paper we will examine how Metacheck, a novel open source tool that checks best practices in data and code sharing, might assist in the process of checking whether data and code are shared. We will compare the output of Metacheck against the manual coding of Cooper et al. (2026b), available at https://github.com/nhcooper123/reproduce-reuse-recycle, which we treat as a careful human comparison rather than as a ground truth that is correct by definition. It is clear that the human coders exerted considerable effort over the 2-day hackathon to code the 1861 papers, where 145 people coded a median of 10 (min = 1, max = 68) manuscripts. Cooper et al.'s protocol covered the seven journals published by the British Ecological Society (BES), coding papers published between 2017 and 2024 on eight groups of questions per paper. The team coded whether data and code were used and archived, where they were archived, whether the archived files could be located, downloaded, and opened, what file formats were used, whether a README was present and how useful it was, how complete the archived data were, how well-

annotated the code was, and whether the data and code were citable (i.e., had a DOI and a licence).

Coding the presence or absence of data and/or code files is exactly the kind of time-consuming task that automated tools such as Metacheck (DeBruine et al., 2026) aim to support. If an automated tool could complete some of Cooper et al.'s eight coding questions at comparable accuracy to human coders, it would substantially reduce the workload that is currently required to check if researchers follow best practices in data and code sharing. Metacheck (DeBruine et al., 2026) is a modular, open-source tool that uses text search, rule-based classification, APIs, and optional large language models to extract information about research practices from scientific manuscripts and their associated repositories. Metacheck's `repo_check`, `data_check`, and `code_check` modules perform automated checks that closely resemble the classifications that Cooper et al. (2026b) asked their human coders to perform.

In this paper, we use Cooper et al.'s openly archived data as the comparison coding (Cooper & the BES Data and Code Hackathon Group, 2026a), and for each of the eight groups of questions in their protocol, we summarize what Cooper et al. found, and examine whether that question can currently be automated with sufficient accuracy. Where the two codings disagree, we had Claude Opus 5.5 review each case to see which is correct, rather than assuming the human coding is right by definition. The results generated by Metacheck are completely automated, do not rely on AI, and can be reproduced by re-running the code. Human input was limited to a few settings, which are recorded in the code: two papers whose repositories hold large archives were given a larger download limit than the 2000 MB used for all other papers, and for one paper, whose repository holds 23 GB of data, the files were listed but not downloaded. Re-running the code also requires access keys for the APIs of GitHub, Dryad, OSF, and Zenodo. Unlike a validation study that reports a single accuracy percentage, our goal here is to make transparent where the current version of Metacheck can reduce the manual burden of this kind of meta-scientific data collection, and where it cannot.

As an automated tool, Metacheck relies on data and code being shared according to the FAIR principles, the idea being that "all research objects should be Findable, Accessible, Interoperable and Reusable (FAIR) both for machines and for people" (Wilkinson et al., 2016, p. 3). Human coders can retrieve information that is not accessible by a machine. For example, if authors share their data as a table in a PDF file in the supplementary material, the data are not FAIR: a table in a PDF cannot be read reliably by a machine, and the file has no persistent identifier of its own. Cooper and colleagues did not limit their coding to FAIR data, and as a consequence, Metacheck will always miss some information that human coders found. On the other hand, machines can find some information that human coders miss. For example, Metacheck found information about licenses, READMEs, and repositories that human coders failed to classify. Throughout this project we will also reflect on practical challenges when developing software that attempts to automatically retrieve FAIR data, and the implications for best practices in data and code sharing.

Methods

We obtained the list of paper DOIs, journals, and years from the cleaned dataset in the public repository, after correcting four DOIs that were wrong in that dataset, and downloaded the articles from the publisher and from Europe PMC with an R script; the full texts are not shared because of copyright. All articles were converted to structured text using GROBID 0.9.0, and converted to Metacheck paper objects. We ran the `repo_check` (locates and lists files in online repositories), `data_check` (classifies files and extracts information about tabular data), and `code_check` (checks R, Python, SAS, SPSS, Stata, Mplus, and MATLAB code files) modules to mirror the manual coding performed by Cooper and colleagues. Subsequently, we ran the `reproducibility_check` module to illustrate additional benefits of automation.

1. Does the paper use data/code? If so, are they archived?

Cooper and colleagues reported that "Of the 1861 papers in our dataset, 93% (n = 1735) used data and 90% (n = 1670) used code. Of these papers, 97% (n = 1690) archived their data, but only 35% (n = 577) archived their code". The first two percentages are based on text in data

availability statements and the method section. There are tools that automatically check data availability statements, such as OddPub (Riedel et al., 2020). Metacheck itself has an equivalent module, `open_practices`, which reads the full text and judges from the wording alone whether open data and/or open code was shared. These automated checks most closely follow the coding by Cooper and colleagues, in that data and code is classified as available irrespective of whether authors followed FAIR principles. Metacheck's `repo_check` module takes a different approach and classifies data and code as available based on a linked repository that can be automatically accessed through an API.

Oddpub and Open Practices Check

OddPub (Riedel et al., 2020) is a tool that searches text for keywords related to data and code sharing. The Metacheck Open Practices module does the same, but is intentionally over-inclusive, which means it is designed to find more data or code sharing statements, while also returning more false positives. We compared the automatic text-based judgements from OddPub and the Open Practices module to Cooper and colleagues' human coding of whether data and code were openly shared (their `data_open`/`code_open` columns), treating "Yes" (and for data "Yes, but not all files") as open and "No" as not open, and excluding papers where the coders used an ambiguous label (e.g., "Needs specific software or too large") or left the item unscored (`data_open`/`code_open` were only asked for papers Cooper and colleagues had already coded as archiving their data/code, and not every such paper has a recorded answer, so the denominators below are smaller than the archiving counts reported earlier). As with the comparison above, this is a comparison against the human coding by Cooper and colleagues, not against a known correct answer, so a disagreement means only that the tool and the human coders reached different judgements. For data sharing (1552 papers with a clear human judgement), OddPub agreed with the human coders on 80% of the papers (307 disagreements: 6 where OddPub judged the data open but the coders did not, and 301 the other way around). Metacheck's `open_practices` module agreed with the human coders on 86% of the papers (220 disagreements: 10 where `open_practices` judged the data open but the coders did not, and 210 the other way around). For

code sharing (498 papers with a clear human judgement), the gap was larger: OddPub agreed 44% of the time (278 disagreements: 1 and 277), whereas `open_practices` agreed 77% of the time (117 disagreements: 3 and 114). The disagreements fall almost entirely on one side: both tools much more often judge a paper to share open data or code than the human coders judge it not to, and this corpus contains very few papers the human coders judged as clearly *not* sharing (11 for data, 3 for code), so the counts of papers the tools agreed were not open rest on very small numbers. `open_practices` and OddPub both determine open data/code from the text of the article alone, but do not examine whether data and code are shared by examining the contents of data repositories, as other Metacheck modules do.

Note that OddPub and `open_practices` classify every paper regardless of how Cooper and colleagues' coders scored this question. For data sharing, 97 papers archived their data (`data_availability == "Yes"`) but were left unscored on whether it could be opened, of which OddPub judged 44 (45%) and `open_practices` judged 78 (80%) to have open data; a further 41 were explicitly flagged by Cooper and colleagues' coders as needing specific software or too large to open, of which OddPub judged 24 (59%) and `open_practices` judged 31 (76%) to have open data. For code sharing, 49 papers archived their code (`code_archived == "Yes"`) but were left unscored on whether it could be opened, of which OddPub judged 8 (16%) and `open_practices` judged 25 (51%) to have open code; a further 34 were explicitly flagged as possibly accessible with the right software, of which OddPub judged 16 (47%) and `open_practices` judged 24 (71%) to have open code.

Repo check, Data check, and Code check

Cooper and colleagues found that 1690 papers archived their data and 577 their code, and in total, 1789 papers archived either data or code. In total, Metacheck found 1561 papers that contained a link to a repository in the main text where it could verify that the repository contained files. Of these, 1487 papers contained data, 452 papers contained code, and 1532 papers contained either data or code according to the Metacheck modules that examine the data and code in repositories. The manually coded classifications agree with the automatic

classification by Metacheck for 1515 papers, and disagree for 291 (274 where Cooper's coders found data or code that Metacheck missed, and 17 papers where Metacheck found data or code that Cooper's coders had not coded as shared).

For Metacheck, data and code is findable and accessible if it is archived in one of the repositories that Metacheck supports (through its support for Dataverse, DSpace 6 and 7, Figshare, and DataOne, and dedicated support for OSF, Zenodo, ResearchBox, Dryad, ReShare, GitHub, GitLab, Mendeley Data, FSD, AsPredicted, and 4TU). Because many repositories use the same underlying API, Metacheck supports 250 unique repositories (121 Dataverse installations, 66 DSpace 7 installations, 6 legacy DSpace 6 installations, 5 DataOne member nodes, and 41 Figshare-based hosts, plus 11 dedicated single-platform integrations), but this is only a subset of all repositories used in manuscripts.

We had Claude Opus 5.5 review the 274 papers where Cooper and colleagues' coders coded data or code as shared and where Metacheck failed to find data or code, one paper at a time, by checking the contents of the repository. This analysis revealed five reasons why Metacheck misses data and code in a repository (see Table 1). In 18 papers we found no reference to a repository or link to a dataset in the manuscript. The data were either explicitly withheld for confidentiality, ethical, or Indigenous-data-sovereignty reasons, could not be shared for legal reasons, or were in shared in the paper or its supplemental material. Some of these papers offer to make data available upon request, but are arguably better classified as papers that explain why data could not be shared. In 81 papers the citation or the repository did not meet basic FAIR requirements, so the data could not be retrieved automatically when we checked. Papers cite a private reviewer or share link instead of the dataset's public identifier (5 papers), or an organisation or user page that identifies no specific deposit (2); or the repository blocks automated access (7), was not live or accessible when checked (14), or offers no API through which a machine can retrieve the data (53). Automated access to repositories has become harder in recent years because of heavy scraping of files, which repository managers assume is mostly done by bots collecting training data for generative AI. In a 2025 survey of 66 repositories, over

90% reported aggressive bot traffic, often causing slowdowns or outages, and many responded with rate limits and other blocks that also stop legitimate machine access (Shearer & Walk, 2025). We observed this directly when attempting to download files for this manuscript.

In 101 papers data is shared on a repository that is FAIR, but Metacheck does not yet support automated downloads from this repository. Metacheck does not yet have support for some field-specific or institutional repositories (for example, the US Geological Survey's ScienceBase, the Environmental Data Initiative, the NERC Environmental Information Data Centre, and PANGAEA, the earth- and environmental-science data publisher). We found 26 papers that shared data or code in a repository Metacheck supports, but in a format Metacheck is not able to automatically classify as data or code. This occurred when data was shared in non-FAIR formats (e.g., a table in a PDF file), or data or code files in a plain .txt file without labeling the file as data or code (which makes it difficult to automatically differentiate from output or documentation files in a repository, while a human can find such data). Another reason was that the file extension is not in the list of files that metacheck classifies as data. For example, drone photographs that are the measurement data in JPG files are classified as image material rather than data by Metacheck. Finally, archives in `.rar` or `.7z` formats can't be opened by R, and Metacheck can not (yet) examine their contents.

For the remaining 48 papers the repository is FAIR and on a platform Metacheck supports, but a technical limitation prevented Metacheck from finding or using it. In 30 papers the PDF to text conversion corrupted the repository identifier, most often by inserting spaces into a DOI (24 papers), or the text produced by GROBID lacked the data availability statement, usually because this section followed the reference list (6). The remaining 18 papers reflect limitations of Metacheck's own code. In 8 of these papers the DOI of the repository appeared only in the reference list, which Metacheck does not search. In 5 papers the authors referred to their data only through a data citation; in the other 3 the authors did give the DOI in the data availability statement, but the conversion from PDF to text moved it into the reference list or scrambled the statement. Data citations allow assigning credit to the reuse of data, and are

therefore good practice, but it is difficult for a machine to identify which DOIs in a reference list point to the data underlying the manuscript. We therefore strongly recommend to always also provide a direct link to the DOI of the repository holding the authors' own data in the main text, for example in a data availability statement. In the remaining 10 papers Metacheck did not follow a Handle link to a repository it supports (4 papers), did not recognise the identifier (a Figshare collection or institutional DOI, 3 papers; an Arctic Data Center identifier that is not a DOI, 1 paper), or did not read files inside an archive (2 papers). The full per-paper record for every one of these 274 papers is available in `data/review/manual_repo_findability_audit_data.csv` and `manual_fair_api_findability_audit_data.csv`.

Table 1

Reasons and frequencies of papers where manual coders found data or code while Metacheck did not.

Cluster	n papers
No repository is cited anywhere in the paper's own text	18
Repository contains data or code that Metacheck did not classify (a format Metacheck does not yet support, or data or code not shared in a FAIR format)	26
Citation or repository is not FAIR (a private or non-specific link, a repository that blocks automated access or is not live, or a platform without an API)	81
Repository is FAIR, but on a platform Metacheck does not support	101
Technical limitation: the repository is FAIR and on a supported platform, but the PDF-to-text conversion or a defect in Metacheck's own code prevented detection or classification	48

In total, 18 papers did not link to a repository, 16 of the 26 papers whose files Metacheck did not classify shared their data or code in a format that is not FAIR, and 81 papers linked to a repository, or used a link, that is not FAIR. This means that 115 of the 274 papers where Cooper

and colleagues' coders found data or code that Metacheck missed are explained by data and code that were not shared following FAIR principles. In other words, of the 1789 papers that Cooper and colleagues' coders coded as sharing data or code, the remaining 1674 shared them in a way that allows an automated tool, in principle, to locate the repository and access the files, and Metacheck found the data or code in 1515 of these, or 90.5%. Resolving technical limitations in text extraction from PDF files, and support for additional repositories would resolve the remaining discrepancies.

2. Where were the data/code archived?

Cooper and colleagues coded the specific repository where data and/or code was stored. They note that 60% of the code files hosted on GitHub or similar platforms were also archived on Zenodo (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). GitHub is not a stable repository, while Zenodo is. Cooper et al. report Dryad as the dominant data repository in the papers they examined (57%, n = 1022), with Figshare (9%), Zenodo (12%), and "various institutional and government repositories" (17%) making up most of the remainder (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959).

Cooper et al. separately coded the repository that stored the data, and the repository that stored the code, and both data and code can be stored in more than one platform per paper. Metacheck does not distinguish between data and code repositories, but lists whether files in all repositories it finds contain data and/or code files. We retrieved the repository address from Metacheck, and assigned each repository to the categories used by Cooper and colleagues (the categories "Supplementary materials" and "Personal website" are not used by Metacheck, as it can only find FAIR repositories). We then classify agreement into five categories: both name the same platforms, Metacheck names fewer platforms, Metacheck names more platforms, each names a platform the other does not, or they share no platform.

Table 2

Agreement between Cooper et al.'s coded archive platform(s) and the platform(s) of the repositories in which Metacheck found data or code, compared as sets of platforms.

Platform	Comparable papers	Same platforms	Metacheck lists fewer	Metacheck lists more	Each lists one the other lacks	No shared platform
Data	1413	1276 (90.3%)	55 (3.9%)	52 (3.7%)	4 (0.3%)	26 (1.8%)
Code	410	320 (78.0%)	56 (13.7%)	22 (5.4%)	2 (0.5%)	10 (2.4%)

For data, 1413 papers could be compared because human coders and Metacheck named at least one repository platform. The two sides named exactly the same platform(s) for 1276 (90.3%) and agreed on at least one shared platform for 1387 (98.2%). Some of the differences are caused by the previously discussed limitations in the repositories that Metacheck supports, but other times Metacheck found more repositories than human coders. In 55 papers Metacheck lists fewer platforms than the coders, most often missing a second repository coded as "Other repo/database" (e.g., an institutional or domain-specific database). In 52 papers Metacheck found an additional platform holding data-classified files that the human coders missed, most often GitHub (40 papers). Usually this is the authors' own repository holding data files alongside the analysis code, but in 9 papers it is a repository of software the authors used, written by someone else.

Metacheck cannot yet differentiate between data or code that authors created and shared, and data or code that they reused, and it treats all files in repositories linked to in a manuscript as the paper's own data or code. Cooper and colleagues asked coders to count data the paper generated or collated, but did not record whether archived files were created by the authors. We asked Claude to examine each of the 215 links to GitHub and GitLab that Metacheck found, and it judged that 31 of these links, in 28 papers, point to a repository whose owner is not an author of the manuscript or their research group. With one exception these hold software or scripts the authors used, not reused data; the exception is the repository of a text corpus, linked to by 2

papers. Authors can make re-use of data or code created by others more transparent and automatically detectable by only using data citations for such repositories. There should not be a direct link to a repository that is re-used in the main text.

3. Can the archived data/code be located, downloaded and opened?

Cooper et al. report that, of the 1735 papers in their dataset that used data, 1690 archived the data, of which 97% could be accessed via the links provided in the data availability statement, 95% could be downloaded, and 91% could be opened (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). Cooper et al.'s coders downloaded data for 1600 papers; for 253 of these papers Metacheck downloaded no data file at all, mostly because it did not find the repository (because the data were not shared in a FAIR way, were in a FAIR repository Metacheck does not support, or because of the technical limitations discussed earlier). Counted over all 1600, Metacheck downloaded and could use the data in 1338 papers (83.6%). For the 1349 papers where both Metacheck and the manual coders gave an answer, they agreed for 1338 (99.2%), and in 11 papers there was a disagreement.

We resolved each of the 11 disagreements by downloading and opening the files. For 5 the manual coders were correct. In 3 the data are space-separated tables, which Metacheck cannot read, and 1 of these also exceeds its 100 MB download limit. In 1 the data are GeoTIFF and shapefile files inside a zip archive. In 1 the data are in NCBI's Sequence Read Archive, a FAIR repository Metacheck does not support. In 6 the review judged Metacheck's answer correct, but this needs care, because Metacheck's answer combines two conditions: the files can be downloaded, and they contain a usable data table. In three papers the data were Excel files with formatted publication tables rather than raw data, and in a fourth the files are model-input files for Stan rather than raw data. In these four papers the files download, as the coders recorded, but they are not usable data tables, which is what Metacheck's answer requires. In the other two the coders' answer was wrong. One is an OSF project that the manual coder reported needing to request access to, although OSF's activity log shows it has been public since January

2021. The other linked to an Excel file for which the manual coder reported a download error, but that Metacheck downloaded without problem.

With respect to code sharing, Cooper et al. report that "only 35% (n = 577) archived their code" among papers using code (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1958), and, among archived code, that most is on Zenodo or GitHub-family platforms (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1959). Cooper et al.'s coders and Metacheck both gave an answer on whether code was archived for 1669 papers. They agreed on 1467 (87.9%) and disagreed on 202 (12.1%). Concerning the question whether shared code could be downloaded, for 404 papers (which is the subset of papers where code was archived) there was almost perfect agreement between manual and automatic classifications (403 of 404 (99.8%), with only 1 disagreement remaining).

We investigated all 203 disagreements about whether whether code was archived (and the one disagreement about whether code could be downloaded). The manual coders were correct in 167 papers and Metacheck was correct in 34 papers, and in 2 papers both were wrong. In 87 papers the code was not shared following FAIR principles. Metacheck could not find the code, but manual coders could. Out of these 87 papers, in 34 authors shared data on Dryad and code on Zenodo, but the paper never directly linked to the Zenodo repository. Instead, the link to Zenodo was only present inside the Dryad repository. Authors should link to code directly in the data availability statement, using the Zenodo DOI. In 33 papers the code is provided only as journal Supporting Information, in 8 it is saved as a `.txt` file or without a file extension, in 5 the paper links to a generic GitHub account of an authors, instead of the specific GitHub repository that contains the shared files, and in 4 the code is inside a Word or PDF document. In the remaining 3 papers the paper gives no link to the code, a private share link, or a GitHub repository that no longer exists.

In the remaining 82 papers the cause lies in earlier mentioned technical limitations of Metacheck, and the manual coders were correct in 80 of them. In 28 papers Metacheck did not detect the repository named in the paper, in 26 the code is in a FAIR repository on a platform

Metacheck does not support, in 13 Metacheck counted software or scripts written by other people as the authors' code, in 7 the code is in a language or file type Metacheck's `code_check` module does not cover, and in 5 it is inside an archive Metacheck does not open. In the remaining 3 papers Metacheck counted a README written in R Markdown as code, or incorrectly classified data and code shared as an R package as software, instead of as the data and code of the paper. In 2 of these 82 papers both sides were wrong. The authors' code is on Zenodo and is linked only from the Dryad page, so the coders' answer that no code was archived is incorrect. Metacheck did not find this code, and its answer that code was archived is a false positive, because it rests only on a file called `README.Rmd`: in one paper in the Dryad deposit, in the other in a repository of another author's R package.

Third, in 34 papers the manual coding was incorrect, and Metacheck's answer was correct. In 21 papers the coders coded that no code was archived, although the authors' code is in the repository. In one of these the repository holds the first author's general solver for the model the paper applies, in the folder of a companion paper, although the paper does not say that it used these scripts. In 12 papers the coders coded code as archived, but we found none. In the remaining paper, which is the one disagreement about downloading code, the coder reported that the link to the code and data did not open. The DOI in the paper leads to an OSF project that has been public since January 2022, before the coding took place, and Metacheck downloaded its R files without problem.

4. What file extensions do the archived data/code files use?

Metacheck can only report the extensions of files it recognises as data or code, so these numbers are not directly comparable to those of Cooper et al. (2026b, p. 1960). Files recognised as data were archived with 158 different file extensions, and of the 2184 unique data file extensions recorded per paper, 86.1% (n = 1880) were saved with the following 10 file extensions: .csv/.tsv, .xls(x), .txt, .rda/.rdata/.rds, .shp, .json, .tre, .tif, .fasta, .dat. Of the 588 unique code file extensions recorded per paper, 79.4% (n = 467) were saved as native source

code, and a further 20.6% (n = 121) as notebook files. Most code in FAIR formats that Metacheck could detect (80.7%, n = 397) was written in R.

5. Do the archived data/code have a README (or equivalent)? If so, how useful is it?

Cooper et al.'s coders were asked whether the data had a "README/metadata", and a README or equivalent was present for 66% (n = 1114) of papers that shared data, and 61% (n = 351) of the papers that shared code (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1960). In total, 69.4% of papers with either data or code included information equivalent to a README. Metacheck checks whether a repository contains a README file, which it determines from the file name (e.g., 'README' or 'data description' in various ways this can be written), or a RO-Crate metadata file. We do not report README quality, because how useful a README is requires a judgement only humans can make. Metacheck found a README file for 53.6% (n = 836) of the 1561 papers with a repository that contains files, which is substantially lower than the manual coders found. The largest cause of disagreement is what counts as a README for Metacheck, compared with the manual coders. In 294 papers the coders recorded a README or equivalent, but the deposit holds no README file. For 270 of these the deposit's web page has a description or abstract, 170 of them on Dryad, and the coders may have counted this as the equivalent of a README (there are no details on what coders counted). We recommend that researchers add a README file to every repository (and preferably a RO-Crate file), as information on websites is typically not accessible through an API, and therefore not FAIR.

6. How complete is the archived data, and can the code be run?

Cooper et al. asked coders to judge how complete the archived data were, but note that "Completeness is difficult to assess thoroughly without reproducing the analyses in the paper, but this requires time and advanced computational skills" (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1957). They report that 69% (n = 1087) of datasets were scored as complete, and that "this should be considered an informed judgement about completeness, not verified reproducibility" (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1960).

Metacheck has code that attempts to automatically reproduce results from data and code in R and Python. We ran the code in a Docker environment that contained the most common pre-installed packages, although this might lead to a difference between the code used to create the script, and the code used to run the script. We set a runtime for each script of at most 3000 seconds, and noted when scripts failed because they needed a longer time to run (e.g., lengthy simulations). When a script reads the output of another script, the module tries to run them in the required order. It changes the code to remove obstacles that are unrelated to the analysis: in 383 papers it rewrote 5627 file paths so that they pointed to the downloaded files, removed 475 `setwd()` calls, added 1696 calls that load a package, and replaced 604 references to fonts. The results below therefore describe what happens after these repairs, and code run exactly as shared would fail more often. Finally, the module extracts the statistical results printed by the code, and looks for the statistics reported in the paper among them. This is a feature still under development, but it attempts to indicate which statistical tests can be found in the results of the reproduced code.

Of the 452 papers in which Metacheck found code, the module tried to run the code of 435. The other 17 papers only have MATLAB or Mplus code, which the reproducibility check module can't run, as this is proprietary software. Not every code file in these papers is a standalone analysis that the module runs on its own, so these files are counted and reported separately rather than among the scripts that were run: 641 files are run by another script through `source()` or an import and have no result of their own, 217 only define functions, 11 are JAGS or BUGS model files, 2 are not code in a language the module runs, and 10 could not be tried because a data file they read was larger than the 1,000 MB per-file limit, and the datafile was not downloaded. Across these 435 papers, 2874 scripts were run as analyses, of which 626 (21.8%) ran without an error (490 of the 2267 R scripts (21.6%) and 136 of the 607 Python scripts (22.4%)). At the level of papers, all scripts ran without an error in only 41 papers (9.4%), some files ran in 135 (31.0%), and none of the files ran in 222 (51.0%).

Table 3 sorts the 2248 scripts that failed, in 357 papers, by the one-word reason the module saved for each script. A paper can appear under several reasons. The module did not start

269 scripts because a package they need could not be installed, and a further 52 files could not be read as code. The other scripts started and then stopped: 1391 with an error, 197 on the time limit, 285 because the code asks a person to choose a file, 21 on the memory limit, 24 because a download they need was too large, and 9 because they need the internet.

Table 3

Reasons why scripts did not run without an error. A paper can appear under several reasons.

Reason	Scripts	% of failed scripts	Papers
A file the script needs was not found	632	28.1	160
A package could not be installed or loaded	376	16.7	76
Another error while the code ran	452	20.1	177
An object or function is not defined	178	7.9	96
The time limit was reached	197	8.8	41
The script asks a person to choose a file	285	12.7	34
The file is not valid code in its language	56	2.5	48
The software environment differs	39	1.7	24
A download the script needs was too large	24	1.1	4
The script needs the internet	9	0.4	7

The most frequent reason is a file the script needs. This can mean that the authors did not archive it (sometimes because the data could not be shared for privacy reasons), that it is produced by a step that is not archived, or that Metacheck could not match the name in the code to a file in the repository (as people sometimes rename files they share after they have run the analyses). In a further 285 scripts the code asks a person to select a file, for example with `file.choose()`, so it cannot run unattended. The next most frequent reason, after 452 scripts that stopped with some other error while the code ran, is a package. The papers declared 4969 packages in total, and 108 of these (2.2%) could not be installed, in 75 papers. In 5 cases the package was not available on CRAN for the version of R the module used. The packages that

failed most often are rethinking (3 papers), SDMTools (3) and countreg (2). Some failures do not concern the authors' analysis at all, because Metacheck treats all files in a linked repository as the paper's own.

These results can be compared with the coders' judgement of completeness. Among the 226 papers whose code was run and whose data the coders rated as complete, the module could not find a file a script needs in 100 papers (44.2%), compared with 177 of all 435 papers (40.7%). At least one script ran without an error in 103 of these papers (45.6%), and all scripts did in 25 (11.1%). The completeness ration is not a valid indication of the computational reproducibility of a paper, Cooper et al. already caution.

This automated computational reproducibility check indicates that creating computationally reproducible data and code is difficult. Running a tool such as Metacheck on a repository before depositing data and code would perhaps have warned the researchers who failed to share all files that some scripts or files were missing, that objects or functions were not defined, or that there were other errors in the code. To prevent problems with packages or dependencies that are no longer available, the best solution is to create a Docker capsule which stores all required software, data, and code to reproduce the analyses. This is the only way to guarantee that scripts can be run years from now. Better coding practices would prevent researchers from creating code that requires human input, or where file paths are not set correctly. Finally, we could perhaps have increased the percentage of successful reproductions by increasing the run-time, and the file size of downloaded files. However, some limits should be set, and ideally scripts are computationally reproducible without downloading very large files, or running code for hours. Although the computational reproducibility rates might have been higher if a human had attempted to reproduce the results, the human coder would - for example - need expertise in python and R, the knowledge how to re-organize repositories, resolve minor coding errors, know which undefined libraries to install, and be able to figure out the order in which files should be run. Most researchers have not received training in computational reproducibility, and our results show there is a lot to gain. Automated checks such as those provided by

Metacheck can reduce common errors, but real long term computational reproducibility might require alternative solutions than just sharing data and code, such as sharing all files in a container that guarantees computational reproducibility. [We should do this ourselves and link to it]

7. How good is the code annotation?

Cooper et al.'s coders rated how well the code was annotated, on a scale from 1 (not annotated at all) to 10 (thorough annotation throughout) (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1957). For the 543 papers they rated, the median score was 7 (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1960), although the authors note that this variable was less consistent among coders and should be interpreted with care (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1958). How good code annotation is cannot be classified automatically: whether a comment explains a decision that is not obvious from the code, or only repeats what the code does, requires a judgement that only human coders can make. Metacheck can check whether code is annotated at all. It counts the comment lines in every code file it can read, and warns when a file has no comments. Of the 8914 code files in 447 papers that Metacheck could check, 956 (10.7%), in 110 papers, had no comments. Of these, 404 were Python files, 303 of which were documented with a docstring instead of comments, which Metacheck reports separately. The median code file had 15 comment lines (interquartile range 3 to 41), against 68 lines of code. Metacheck can not evaluate annotation quality, but it can warn when code is not annotated at

8. Are the data/code citable?

Cooper et al. report that "of the 1690 papers with archived data, 91% (n = 1543) had a DOI and 85% (n = 1438) had a licence", with CC0, the licence used by Dryad, used most often (n = 1076), and that "of the 577 papers with archived code, 79% (n = 457) had a DOI and 74% (n = 428) had a licence", with CC BY (n = 122), the licence Zenodo uses by default, and CC0 (n = 104) used most often (Cooper & the BES Data and Code Hackathon Group, 2026b, p. 1960). Metacheck reads the licence from the metadata of each repository it can access, and we

compared whether the data had a licence. For the 1333 papers where both the manual coders and Metacheck gave an answer, they agreed for 1276 papers (95.7%) and disagreed for 57 (Table 4). In 50 of these disagreements Metacheck found a licence that the coders did not record, and the review of each disagreement judged the automatic coding correct in 46 of the 57. We did not compare DOIs or the licences of code.

Discussion

Researchers should share data and code following FAIR principles where possible. Manually checking whether data and code is shared correctly is time-consuming. We explored the extent to which automated tools can identify whether data and code is shared, and if best practices (e.g., including a README) are adopted. If automated tools can perform this task accurately, the workload for editors or peer reviewers who want to check whether submissions follow open science principles can be reduced substantially.

Table 4 summarizes how well Metacheck, a module based open source tool that can check data and code repositories for best practices, can identify data and code, the presence of a README and license, compared to the manual classifications of a team of 145 participants of a hackathon. The results show that in a substantial percentage of papers an automated check is able to identify the presence of downloadable data and code in a repository. Metacheck agrees with the classifications by manual coders for more than 85% of data and code repositories. In some of the disagreements, Metacheck can correctly identify data and code that manual coders missed. Other disagreements are due to the fact that Metacheck is designed to only identify FAIR data and code, while manual coders also classified non-FAIR repositories, and due to technical limitations of Metacheck (e.g., imperfect extraction of text from PDF files, unsupported repositories, imperfect processing of some data and code file-types).

Table 4

Agreements and disagreements between manual and automatic coding of data and code sharing practices.

Construct	n	Agree %	Disagreements	Manual coding correct	Automatic coding correct	Different definition	Both defensible	Both wrong
Data availability	1735	85.5	251	227	24	0	0	0
Data licence present	1333	95.7	57	8	46	0	3	0
Data downloadable and usable	1349	99.2	11	5	6	0	0	0
Code archived	1669	87.9	202	167	33	0	0	2
Code downloadable	404	99.8	1	0	1	0	0	0
Any README present	1726	72.9	467	156	17	294	0	0

The current results indicate that Metacheck can assist in the classification of data and code, especially when data and code is shared following FAIR principles. When Metacheck can't find data or code, it might still be shared, and in the 15% of cases where Metacheck missed data or code, manual coding remains necessary. Furthermore, Metacheck is currently unable to distinguish a GitHub link to a repository that shared the authors' code from a link to existing software that was used by the authors. Manual checks will remain necessary, and Metacheck is designed to assist, not replace human judgment. Despite several imperfections in the current version of Metacheck, we hope it can substantially reduce the time that scientists need to check whether articles share data and code.

Future research could improve both how Metacheck works, as how people share data and code. First, PDF files are created to optimize human readability, not machine readability. Text extraction from PDF files has known limitations which lead to broken or undetected links to repositories. It might be possible to remedy this by developing software that improved text

extraction from PDF files (e.g., bibr, Werner, 2026), but a fully machine-readable solution would require authors or publishers to share the machine readable XML files used to generate manuscripts, or to add metadata files alongside journal publications with important information (such as links to data and code). Second, automated tools should eventually support all FAIR repositories. Given the diversity of data and code storage solutions across disciplines, this will require collaboration between tool builders, research communities, and data repository managers. Third, automated tools should detect all file formats that store data and code, which once again given the diversity of research software used across disciplines requires collaboration between tool builders and research communities.

The comparison against the manually coded papers also highlights aspects where researchers, journals, and repositories can improve. Researchers should follow best practices in machine readable FAIR data and code sharing (Lakens et al., 2026), and journals should ask authors to follow FAIR principles. Automated tools such as Metacheck can assist in such checks, as the current manuscript demonstrates. Repositories should be FAIR, and accessible by a machine. The Dryad repository does not allow downloads of file contents without an API key, and Dryad has stricter limits on the number of API calls, which severely limits automated checks. Researchers are recommended to share data in fully machine accessible repositories, such as Zenodo, instead.

The manually coded data be Cooper et al. (2026b) was extremely valuable to improve Metacheck. Over the course of preparing this manuscript we carefully compared the automated results of Metacheck against the manually coded results by Cooper and colleagues. This led us build in support for the backends of data repositories (e.g., Dataverse, DSpace, DataOne, and Figshare) where we previously supported only specific instances of data repositories, raising support from 6 data repositories to over 250. Numerous changes and improvements were made to Metacheck to allow it to support a wider range of data and code formats, and deal better with the the way researchers currently share datasets. For this reason, we believe projects where the scientific literature is systematically coded across different disciplines for features that

researchers in these communities deem valuable will continue to be extremely important to develop tools like Metacheck. Appendix A documents in full detail how Metacheck improved over the course of this project.

It is essential that tools can automatically retrieve the data and code underlying scientific manuscripts to achieve the full potential these research outputs. Metacheck offers researchers and journals an easy way to check if data and code are shared and follow FAIR principles. As open science practices increase the demands on researchers, automated tools can assist and reduce the burden, while increasing the quality and value of scientific research.

References

Cooper, N., & the BES Data and Code Hackathon Group. (2026a). *Data and code for MEE paper v1.0*. Zenodo. https://doi.org/10.5281/zenodo.19737911

Cooper, N., & the BES Data and Code Hackathon Group. (2026b). Data- and code-archiving in the British Ecological Society journals: Present status and recommendations for future improvements. *Methods in Ecology and Evolution*, *17*, 1954–1966. https://doi.org/10.1111/2041-210x.70338

Crüwell, S., Apthorp, D., Baker, B. J., Colling, L., Elson, M., Geiger, S. J., Lobentanzer, S., Monéger, J., Patterson, A., Schwarzkopf, D. S., Zaneva, M., & Brown, N. J. L. (2023). What's in a badge? A computational reproducibility investigation of the open data badge policy in one issue of psychological science. *Psychological Science*, *34*(4), 512–522. https://doi.org/10.1177/09567976221140828

DeBruine, L., Mesquida, C., Werner, J., & Lakens, D. (2026). *Metacheck: Check research outputs for best practices*. https://github.com/scienceverse/metacheck

Hamilton, D. G., Page, M. J., Finch, S., Everitt, S., & Fidler, F. (2022). How often do cancer researchers make their data and code available and what factors are associated with sharing? *BMC Medicine*, *20*(1), 438. https://doi.org/10.1186/s12916-022-02644-2

Lakens, D., Mesquida, C., & DeBruine, L. (2026). *Best practices for machine readable data and code sharing in psychology*. https://github.com/scienceverse/ms_FAIR_data_and_code

Li, W., Liu, X., Zhang, Q., Shi, L., Zhang, J.-X., Zhang, X., Luan, J., Li, Y., Xu, T., Zhang, R., Han, X., Lei, J., Wang, X., Wang, Y., Lan, H., Chen, X., Wu, Y., Wu, Y., Xia, L., … Chen, Z. (2026). Formalistic data and code availability policy in high-profile medical journals and pervasive policy-practice gaps in published articles: A meta-research study. *Accountability in Research*, *33*(2), 2481943. https://doi.org/10.1080/08989621.2025.2481943

Miske, O., Abatayo, A. L., Daley, M., Dirzo, M., Fox, N., Haber, N., Hahn, K. M., Struhl, M. K., Mawhinney, B., Silverstein, P., Stankov, T., Tyner, A. H., Adamkovič, M., Alzahawi, S.,

Anafinova, S., Awtrey, E., Axxe, E., Bailey, J., Bakker, B. N., … Errington, T. M. (2026). Investigating the reproducibility of the social and behavioural sciences. *Nature*, *652*(8108), 126–134. https://doi.org/10.1038/s41586-026-10203-5

Obels, P., Lakens, D., Coles, N. A., Gottfried, J., & Green, S. A. (2020). Analysis of open data and computational reproducibility in registered reports in psychology. *Advances in Methods and Practices in Psychological Science*, *3*(2), 229–237. https://doi.org/10.1177/2515245920918872

Riedel, N., Kip, M., & Bobrov, E. (2020). ODDPub – a Text-Mining Algorithm to Detect Data Sharing in Biomedical Publications. *Data Science Journal*, *19*(1). https://doi.org/10.5334/dsj-2020-042

Shearer, K., & Walk, P. (2025). *The impact of AI bots and crawlers on open repositories: Results of a COAR survey*. Confederation of Open Access Repositories (COAR). https://coar-repositories.org/wp-content/uploads/2025/06/Report-of-the-COAR-Survey-on-AI-Bots-June-2025-1.pdf

Trisovic, A., Lau, M. K., Pasquier, T., & Crosas, M. (2022). A large-scale study on research code quality and execution. *Scientific Data*, *9*(1), 60. https://doi.org/10.1038/s41597-022-01143-6

Vilhuber, L., & Cavanagh, J. (2025). Report of the AEA Data Editor. *AEA Papers and Proceedings*, *115*, 944–957. https://doi.org/10.1257/pandp.115.944

Werner, J. (2026). *bibr: Scientific paper extraction to structured JSON* (Version 0.6.0) [Computer software]. https://github.com/scienceverse/bibr/releases/tag/v0.6.0

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, *3*(1), 160018. https://doi.org/10.1038/sdata.2016.18